

Modifiche IPA — AOT-literal universale & Input Vector flessibile

Refactor del design space P1/P2/P3 · documento delle modifiche a partire dal Task 1 · 2026

Stato di verifica. Tutto il codice è stato scritto e verificato **a livello Python** (sintassi, generazione sorgenti, risoluzione descrittore, dimensioni struct). **Non** è stato compilato né caricato nel kernel: l'ambiente di sviluppo è Windows, mentre eBPF/BCC/clang-bpf/verifier girano solo su Linux. La verifica end-to-end (compilazione BCC, verifier, `test_suite`) va fatta sul box Linux. I punti che lo richiedono sono segnalati come *“da verificare su Linux”*.

Sommario delle quattro attività

#	Attività	Esito
1	AOT-literal universale per hardcoded (P1)	FATTO
2	AOT “semplice” per template/modular	SALTATO
3	Input Vector flessibile a runtime per template (P2) e modular (P3)	FATTO
4	Pulizia codice obsoleto	FATTO

Il concetto chiave che ha guidato le scelte. Due assi *ortogonali*: **pesi** (literal in P1 · in mappa in P2/P3) e **compilazione** (AOT offline · BCC a runtime). “AOT-literal” = literal $\wedge$ AOT, specifico di P1. Non è applicabile ai map-based senza snaturarli (Task 2). Invece la **flessibilità dell'IV** (descrittore per-modello) è ortogonale e si porta ovunque (Task 3).

Task 1 — AOT-literal universale (hardcoded P1) FATTO

Problema di partenza

Il generatore AOT offline (`gen_full_c.py` , dialetto libbpf) era **cablato** al solo descrittore default `65-4-4-7` . Qualsiasi descrittore sparse/eterogeneo era supportato solo dal path BCC (`ebpf_program.py`), non dall'AOT. Un guard rifiutava i descrittori non-default.

Cosa è cambiato

`gen_full_c.py` è ora **descrittore-driven**: i 3 *kind* di feature sono stati portati dal dialetto BCC al dialetto libbpf, riproducendo la stessa logica di `ebpf_program._gen_feature_*` :

scalar	-> ttl = ip->ttl	(dal pacchetto)
dense_vector_map	-> ls0..lsN via 1 bpf_map_lookup_elem	(link_state / queue_state)
onehot	-> un solo switch che seleziona 1 peso	(ingress_iface / node)

Ogni descrittore accettato dal BCC è ora anche **AOT-compilabile**. Le mappe dense (link_state , queue_state) vengono emesse solo se il descrittore le usa, dimensionate dalla topologia.

File modificati

File	Modifica
shared/poc_aot/gen_full_c.py	Riscritto descrittore-driven: _feat_scalar/_feat_dense_vector/_feat_onehot_* , _emit_maps() per-descrittore, _resolve_shape() via model_meta.derive_shape .
shared/methods/method4_hardcoded_aot.py	Rimosso il guard _check_supported / _SUPPORTED ; passa meta + topology_config al generatore.
shared/poc_aot/loader_aot.c	Seeding generico: seed_vec_map() semina link_state / queue_state solo se presenti (dimensione dalla mappa); classi mac_table derivate da n_out .

Come funziona adesso

- **Default** [link_state, ingress_iface, ttl, node] → 65-4-4-7 **byte-identico** a prima (stessi pesi, stesse colonne; ora combacia col dialetto BCC).
- **Custom** es. [link_state, queue_occupancy, ttl] → n_in=11 , emette mappa queue_state , qs0..qs3 , niente node/iface.
- Il path **BCC resta intatto** come fallback garantito.

Verificato (Python). Diff generatore nuovo vs vecchio sul default: solo differenze cosmetiche + riordino commutativo dei termini, *pesi e colonne identici*. Descrittore custom: mappa queue_state emessa, slicing pesi corretto (bias a offset 44 per 11-4-4-7).

Task 2 — AOT “semplice” per template/modular SALTATO

Sconsigliato e non implementato. Motivo: **beneficio marginale per design**. Template e modular compilano **una sola volta** all'avvio (il programma è indipendente dal modello; nuovo modello = solo bpf_map_update_elem). L'AOT toglierebbe una singola compilazione di startup, non i “1660 ms per modello” che affliggono solo P1. In più richiederebbe riscrivere due programmi interi dal dialetto BCC a libbpf. Costo alto, ritorno quasi nullo.

Task 3 — Input Vector flessibile a runtime (P2 template, P3 modular)

FATTO

Problema di partenza

Le pipeline map-based avevano l'IV **cablato** al layout protocollare a 65 (link_state 0-5, iface one-hot 6-11, ttl 12, node one-hot 13-64), con offset fissi nel C. I pesi erano già in mappa (nessuna ricompilazione per modello) ma il *layout delle feature* era rigido: tutti i modelli dovevano usare le stesse 4 feature nello stesso ordine.

Decisioni di design (confermate)

Decisione	Scelta
Sorgente del descrittore a runtime	Registry map <code>model_desc[model_id]</code> , popolata dal control plane. NON dal pacchetto (autorevole, niente fiducia nel traffico, zero costo/pacchetto extra oltre 1 lookup).
Ceilings di unroll	<code>MAX_FEAT=4</code> . Le dimensioni dense sono topology-fixed (link_state=6, queue=4); i one-hot grandi (node 52) sono un <i>singolo lookup runtime-indexed</i> , non srotolati.
Tecnica di loop	<code>#pragma unroll</code> (portabile, stile esistente; nessuna dipendenza da kernel ≥ 5.13).
Coppie <code>feat*</code> dell'header IPA	Restano metadata/osservabilità , NON sorgente dell'IV.

La nuova registry `model_desc`

```
#define MAX_FEAT 4
struct feat_ent { __u8 code; __u8 size; __u8 col_off; __u8 _pad; };
struct model_desc { __u8 n_feat; __u8 n_in; __u8 p0; __u8 p1;
                    struct feat_ent feats[MAX_FEAT]; }; /* 20 byte */
BPF_HASH(model_desc, __u8 model_id, struct model_desc, 256);
```

`col_off` = colonna iniziale della feature nella riga di input di fc1 (somma cumulativa delle size precedenti). Il control plane la calcola con `model_meta.resolve_descriptor()` , così l'IV a runtime coincide col layout dei pesi allenati.

Il primo layer, ora descrittore-driven

Gli offset cablati (`+12` ttl, `+13` node, `5+iface`) sono sostituiti da un loop unrolled su `MAX_FEAT` che, per ogni neurone `j` , accumula il contributo di ogni feature dichiarata alla sua colonna runtime:

```

for (f = 0; f < MAX_FEAT; f++) if (f < desc->n_feat) {
    base = woff + fc1_w_off + j*n_in + desc->feats[f].col_off;
    switch (code) {
        TTL            -> acc += _ttl * W(base);
        LINK_STATE     -> for i<size: acc += ls[i] * W(base+i);    // unroll 6
        QUEUE_OCC      -> for i<size: acc += qs[i] * W(base+i);    // unroll 4
        INGRESS_IF     -> if active: acc += W(base + (iface-1));    // 1 lookup
        NODE           -> if node<size: acc += W(base + node);     // 1 lookup
    }
}

```

`n_in` è ora runtime (da `model_desc`) e guida il layout dei pesi (`fc1_b_off = n_in*n_h1`, ecc.). Gli indici restano *bounded* (il check `idx >= MAX_WEIGHT_ENTRIES` / il `lookup` che ritorna NULL) → verifier-safe.

File modificati

<code>shared/model_meta.py</code>	Aggiunti <code>FEATURE_CODE</code> (codici numerici, sorgente unica: <code>link_state=1, iface=2, ttl=3, node=4, queue=5</code>) e <code>resolve_descriptor()</code> (→ lista <code>{code,size,col_off}</code>).
<code>shared/ebpf_template_arch.py</code>	Mappe <code>model_desc</code> + <code>queue_state</code> ; <code>fc1</code> descrittore-driven; <code>n_in</code> runtime nel layout pesi; <code>arch_weight_count(n_h1,n_h2,n_in)</code> parametrizzato; <code>load_model_desc()</code> ; <code>load_arch_weights()</code> semina <code>model_desc</code> (default) internamente.
<code>shared/ebpf_modular.py</code>	Stesse mappe/strutture nel common header; <code>layer_first</code> descrittore-driven; check first-layer <code>n_in==65</code> rilassato a <code>[1,128]</code> ; <code>load_model_desc()</code> ; <code>load_modular_weights()</code> semina <code>model_desc</code> internamente.
<code>shared/methods/method5_template.py</code> , <code>method6_modular.py</code>	Nessuna chiamata extra necessaria (il seeding è integrato nei loader dei pesi).

Verificato (Python). Il descrittore default risolve a `[(1,6,0),(2,6,6),(3,1,12),(4,52,13)]`, `n_in 65` → **esattamente** gli offset cablati vecchi (comportamento byte-identico). Descrittore custom `[link_state,queue,ttl]` → `col_off 0/6/10`. Struct `model_desc` = 20 byte (combacia col C). Conteggio mappe: template 2/2 (dispatcher + ramo standalone), modular 1/1 (common header incluso una volta). Zero offset cablati residui.

Compatibilità test harness. Il seeding di `model_desc` è stato **integrato dentro** `load_arch_weights()` e `load_modular_weights()`: così *tutti* i chiamanti esistenti (i metodi e i test: `verify_prog_run`, `verify_multi_model`, `bench_model_add`, `test_suite`) registrano il descrittore default senza modifiche. Senza questo, l'inferenza avrebbe restituito `XDP_PASS` (lookup fallito).

Nota sulla narrativa design-space

P2/P3 diventano **descrittore-flessibili** come P1: non più "65 fisso". Da concordare col relatore come impatta il confronto della tesi.

Task 4 — Pulizia obsoleti FATTO

Codebase risultato pulito. Eliminati 3 alias di retrocompatibilità **morti** (0 usi) in `shared/model_meta.py`, residui del refactor `node_config` → `topology_config`:

- `DEFAULT_NODE_CONFIG` (alias di `DEFAULT_TOPOLOGY_CONFIG`)
- `load_node_config` (alias di `load_topology_config`)
- `node_config_for` (alias di `topology_config_for`)

Il resto (`method4/5/6`, `test_ipa`, `fix_bpf.sh`, `host_headers`, `weights_method2.json`) è attivo e conservato. Rimosso anche il guard obsoleto `_check_supported` dell'AOT (parte del Task 1).

Come testare (sul box Linux)

```
# Task 1 – AOT-literal universale
python3 shared/methods/method4_hardcoded_aot.py          # default 65-4-4-7
# + con un model_meta.json custom accanto al .pt per un descrittore sparse

# Task 3 – P2/P3 con IV descrittore-driven (default => output invariato)
sudo python3 shared/execute_pipeline.py --method template --iface eth0 --model-id 0
sudo python3 shared/execute_pipeline.py --method modular  --iface eth0 --model-id 0

# Regressione / verifier / perf
sudo python3 shared/test/test_suite.py --kernel
sudo python3 shared/test/verify_prog_run.py
sudo python3 shared/test/verify_multi_model.py
```

Cosa controllare per primo su Linux. (1) Che i tre programmi **passino il verifier** col nuovo primo layer unrolled (budget complessità). (2) Che l'output del descrittore **default** sia identico a prima (regressione byte-compatibile). (3) Se un descrittore usa `queue_occupancy`, seminare `queue_state` via `queue_state_monitor.init_queue_state()` nel control plane.

Riepilogo in una riga

L'**AOT-literal** non è più bloccato al 65-4-4-7 (P1), e l'**Input Vector** è ora **descrittore-driven a runtime** anche nei map-based P2/P3 (via registry `model_desc`), con il descrittore default byte-compatibile ovunque. Task 2 saltato (non conveniente per design), obsoleti rimossi. Tutto scritto e verificato in Python; **compilazione e verifier da eseguire su Linux**.